VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Faculty of Electrical and Electronics Engineering

ASIC Verification

EE5213

Exercise 1 Report

Counter Design and Parameterized Stack Verification

Student 1: Vu Tien Giang - 2570188

Instructor: TS. Tran Hoang Linh

February 28, 2026

1 Exercise 1: Counter Design and Verification

1.1 Objective

Design a 5-bit synchronous counter based on the specifications provided and write a testbench to verify its correctness against a target timing trace.

1.2 Design Specifications

- The counter is clocked on the rising edge of `clk`.
- `rst` is active high and asynchronously resets the output to 0.
- `cnt_in` and `cnt_out` are 5-bit signals.
- If `load` is high, the counter is loaded from `cnt_in` (takes priority over `enab`).
- Otherwise, if `enab` is high, `cnt_out` is incremented.

1.3 Implementation (`counter.v`)

```
`timescale 1ns / 1ps

module counter (
    input clk,
    input rst,
    input load,
    input enab,
    input [4:0] cnt_in,
    output reg [4:0] cnt_out
);
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            cnt_out <= 5'b00000;
        end else if (load) begin
            cnt_out <= cnt_in;
        end else if (enab) begin
            cnt_out <= cnt_out + 1;
        end
    end
endmodule
```

1.4 Testbench (`counter_tb.v`)

To match the precise timestamps detailed in the exercise, the testbench drives inputs securely synchronized around the clock edges.

```
`timescale 1ns / 1ps

module counter_tb;
    reg clk;
    reg rst, load, enab;
    reg [4:0] cnt_in;
    wire [4:0] cnt_out;

```

```verilog
    counter uut (.clk(clk), .rst(rst), .load(load), .enab(enab), .cnt_in(cnt_in), .cnt_out(cnt_out));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1; load = 0; enab = 0; cnt_in = 0;

        #12; rst = 0; load = 1; enab = 1; cnt_in = 5'b10101;
        #8;  $display("At time %0t rst=%b load=%b enab=%b cnt_in=%b cnt_out=%b",
             $time, rst, load, enab, cnt_in, cnt_out);

        #2;  rst = 0; load = 1; enab = 1; cnt_in = 5'b01010;
        #8;  $display("At time %0t rst=%b load=%b enab=%b cnt_in=%b cnt_out=%b",
             $time, rst, load, enab, cnt_in, cnt_out);

        #2;  rst = 0; load = 1; enab = 1; cnt_in = 5'b11111;
        #8;  $display("At time %0t rst=%b load=%b enab=%b cnt_in=%b cnt_out=%b",
             $time, rst, load, enab, cnt_in, cnt_out);

        #2;  rst = 1; load = 1; enab = 1; cnt_in = 5'b11111;
        #8;  $display("At time %0t rst=%b load=%b enab=%b cnt_in=%b cnt_out=%b",
             $time, rst, load, enab, cnt_in, cnt_out);

        #2;  rst = 0; load = 1; enab = 1; cnt_in = 5'b11111;
        #8;  $display("At time %0t rst=%b load=%b enab=%b cnt_in=%b cnt_out=%b",
             $time, rst, load, enab, cnt_in, cnt_out);

        #2;  rst = 0; load = 0; enab = 1; cnt_in = 5'b11111;
        #8;  $display("At time %0t rst=%b load=%b enab=%b cnt_in=%b cnt_out=%b",
             $time, rst, load, enab, cnt_in, cnt_out);

        #10; $finish;
    end
endmodule
```

1.5 Simulation Results

The simulation log matches the specified requirements at the detailed time points.

```
At time 20000 rst=0 load=1 enab=1 cnt_in=10101 cnt_out=10101
At time 30000 rst=0 load=1 enab=1 cnt_in=01010 cnt_out=01010
At time 40000 rst=0 load=1 enab=1 cnt_in=11111 cnt_out=11111
At time 50000 rst=1 load=1 enab=1 cnt_in=11111 cnt_out=00000
At time 60000 rst=0 load=1 enab=1 cnt_in=11111 cnt_out=11111
At time 70000 rst=0 load=0 enab=1 cnt_in=11111 cnt_out=00000
```

2 Exercise 2: Stack Design and Verification

2.1 Objective

Create a parameterized LIFO (Last-In-First-Out) Stack design, present testing scenarios, and write a testbench to verify edge cases.

2.2 Design Specifications

- Parameterized for `WIDTH` and `DEPTH`. `DEPTH` is dynamically managed via `DEPTH_LOG2` (power of 2 constraint).
- Asynchronous high-active reset (`rst`).
- `push` (write) and `pop` (read) synchronous to the rising edge of `clk`.
- Data written later corresponds to earlier reads (LIFO constraint).
- Ignores push attempts when `full` flag is set.
- Ignores pop attempts when `empty` flag is set.

2.3 Implementation (`stack.v`)

```verilog
`timescale 1ns / 1ps

module stack #(
    parameter WIDTH = 8,
    parameter DEPTH_LOG2 = 3
) (
    input clk, rst, push, pop,
    input [WIDTH-1:0] din,
    output reg [WIDTH-1:0] dout,
    output reg full, empty
);
    localparam MAX_DEPTH = 1 << DEPTH_LOG2;
    reg [WIDTH-1:0] mem [0:MAX_DEPTH-1];
    reg [DEPTH_LOG2:0] sp;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            sp <= 0; full <= 0; empty <= 1; dout <= 0;
        end else begin
            if (push && !full) begin
                mem[sp] <= din;
                sp <= sp + 1;
                empty <= 0;
                if (sp + 1 == MAX_DEPTH) full <= 1;
            end else if (pop && !empty) begin
                dout <= mem[sp - 1];
                sp <= sp - 1;
                full <= 0;
                if (sp - 1 == 0) empty <= 1;
            end
        end
```

```
    end
endmodule
```

2.4 Testbench (stack_tb.v)

```
`timescale 1ns / 1ps

module stack_tb;
    parameter WIDTH = 8;
    parameter DEPTH_LOG2 = 2; // Testing depth 4

    reg clk, rst, push, pop;
    reg [WIDTH-1:0] din;
    wire [WIDTH-1:0] dout;
    wire full, empty;

    stack #(.WIDTH(WIDTH), .DEPTH_LOG2(DEPTH_LOG2)) uut (
        .clk(clk), .rst(rst), .push(push), .pop(pop),
        .din(din), .dout(dout), .full(full), .empty(empty)
    );

    initial forever #5 clk = ~clk;

    initial begin
        rst = 1; push = 0; pop = 0; din = 0;
        #15; rst = 0; #10;

        $display("Pushing elements...");
        push_data(8'h11); push_data(8'h22); push_data(8'h33); push_data
    (8'h44);

        $display("Trying to push when full...");
        push_data(8'h55);
        if (full) $display("Stack is full, correctly blocked push.");

        $display("Popping elements...");
        pop_data(); pop_data(); pop_data(); pop_data();

        $display("Trying to pop when empty...");
        pop_data();
        if (empty) $display("Stack is empty, correctly blocked pop.");

        #20 $finish;
    end

    task push_data(input [WIDTH-1:0] val);
    begin
        @(posedge clk); push = 1; din = val;
        @(posedge clk); push = 0;
        $display("Pushed %h | full=%b empty=%b", val, full, empty);
    end
    endtask

    task pop_data();
    begin
        @(posedge clk); pop = 1;
        @(posedge clk); pop = 0;
```

```
        $display("Popped %h | full=%b empty=%b", dout, full, empty);
    end
    endtask
endmodule
```

2.5 Simulation Results

```
Pushing elements...
Pushed 11 | full=0 empty=0
Pushed 22 | full=0 empty=0
Pushed 33 | full=0 empty=0
Pushed 44 | full=1 empty=0
Trying to push when full...
Pushed 55 | full=1 empty=0
Stack is full, correctly blocked push.
Popping elements...
Popped 44 | full=0 empty=0
Popped 33 | full=0 empty=0
Popped 22 | full=0 empty=0
Popped 11 | full=0 empty=1
Trying to pop when empty...
Popped 11 | full=0 empty=1
Stack is empty, correctly blocked pop.
```